

ЛАБОРАТОРНАЯ РАБОТА № 13

ЗАЩИТА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ МЕТОДАМИ СТЕГАНОГРАФИИ

Цель работы: ознакомление с методами защиты исполняемых файлов. Изучение возможностей стеганографической защиты .exe файлов путем встраивания цифровых водяных знаков в пустое место, в конце секции файла.

Примечание. Для выполнения лабораторной работы на компьютере необходимо установить программу *Filigrana.exe*

Описание лабораторной работы. *Защита программного обеспечения (ПО)* — комплекс мер, направленных на защиту ПО от несанкционированного приобретения, использования, распространения, модифицирования, изучения и воссоздания аналогов.

Разработка наиболее эффективного метода защиты для того или иного программного продукта в настоящее время становится одной из важных задач большинства программистов, которые занимаются разработкой специализированного, платного программного обеспечения, так как это позволяет им продавать свой интеллектуальный труд и исключить возможности его нелегального использования среди потребителей, говоря иными словами, никто не сможет использовать оригинальную, лицензионную копию определенной программы, предварительно не купив, не заплатив денег ее разработчику.

Затраты производителей на создание эффективного метода защиты их программных продуктов окупаются и компенсируют потенциальный ущерб, наносимый нелегальным копированием и использованием программ.

Существуют два основных способа защиты интеллектуальной собственности и, следовательно, самих программных продуктов:

1) *юридический* — данный способ защиты заключается в создании определенных законодательных актов, которые будут охранять интеллектуальную собственность (в нашем случае программные продукты) от нелегального использования;

2) *технический* — реализуется путем включения в программный продукт какого-либо из существующих методов защиты, который будет запрещать его нелегальное использование. По сравнению с юридическим способом защиты он является наиболее распространенным, так как он практичен и сравнительно недорогой в реализации.

Юридическая защита. Юридическая защита включает в себя такие методы как патентование, оформление авторских прав на интеллектуальную собственность и т.д. Также предусматривается возможность лицензирования программного продукта, так, например большинство программных продуктов поставляются вместе с лицензией, которая подтверждает право пользователя использовать этот программный продукт, т.е., покупая лицензионную копию программы, пользователь в некой мере производит покупку лицензии на право работы с ее копией.

Техническая защита. Существует множество технических методов защиты программных продуктов. Ниже будут рассмотрены наиболее интересные и актуальные из них.

Использование парольной защиты (кода активации). Этот вариант на сегодня является самым распространенным. Принцип действия защиты основан на идентификации и аутентификации пользователя ПО путем запроса у него дополнительных данных, это могут быть название фирмы и (или) имя и фамилия пользователя и его пароля либо только пароля. Эта информация может запрашиваться в различных ситуациях, например при старте программы, по истечении срока бесплатного использования ПО, при вызове процедуры регистрации или в процессе установки на компьютер пользователя. Процедура защиты проста в реализации и поэтому очень часто применяется производителями ПО. Она сводится к проверке правильности вводимого кода и запуске или незапуске ПО в зависимости от результатов проверки.

Защита через Интернет. Данный метод защиты схож с предыдущим, его отличие заключается в том, что проверка кода активации происходит на сервере производителя. Если ваша программа работает с сетью (это может быть менеджер закачки файлов, браузер, FTP-клиент и пр.), можно применить проверку введенного в программу кода через Интернет, используя базу данных пользователей на сайте программы. Однако проверять нужно какие-либо косвенные данные, чтобы исследователь не мог получить с сайта программы, например, базу данных с серийными номерами.

Сложность взлома этой защиты в том, что весьма непросто найти в коде программы место, где идет обращение к Интернет-серверу именно с целью проверки регистрационного кода.

Выполнение программ на стороне сервера. Данный метод защиты основан на технологии клиент-сервер, он позволяет предотвратить отсылку кода программы пользователям, которые будут с ней работать, так как сама программа хранится и выполняется на сервере, а пользователи, используя клиентскую часть этой программы, получают результаты ее выполнения (рис. 4.1).

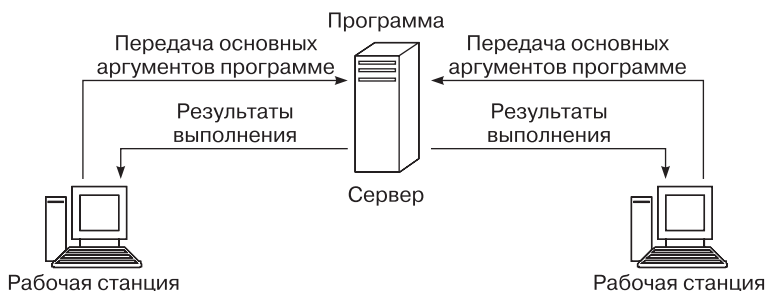


Рис. 4.1. Выполнение программы на стороне сервера

Клиентскую часть можно распространять бесплатно, это позволит пользователям платить только за использование серверной части.

Недостатком данного метода является то, что он устанавливает зависимость пропускной способности сети и тех данных, с которыми будет работать программа (соответственно работа с мультимедиа данными требует максимальной пропускной способности сети). Поэтому данный метод наиболее эффективен для простых программ (сценариев), потребность в которых очень велика среди пользователей.

Преимущество такого метода заключается в том, что злоумышленнику в данном случае нужно будет для начала скомпрометировать сам сервер, только после чего он сможет получить копию требуемой программы.

Данный метод не позволяет защитить программу от нелегального копирования, поэтому после того как ее копия попадет к злоумышленнику, он сможет делать с ней что захочет.

Установка подлинности кода (tamper-proofing). В данном случае в программу помещается процедура проверки целостности самой программы, что позволяет определить, была ли программа изменена (были ли внесены какие-либо изменения в ее код). Если эта процедура обнаруживает, что в программу внесены изменения, она делает программу нефункциональной (рис. 4.2).

Это позволяет защитить программный продукт от модификации со стороны злоумышленника.

Существуют такие пути проверки целостности программы, как:

- проверка идентичности оригинальной и запускаемой программы. Обычно для этого определяется контрольная сумма запущенной программы, которая потом сверяется с записанной в процедуру проверки контрольной суммой оригинальной программы; для осуществления быстрой проверки используют такие алгоритмы как CRC или MD4/5;

- проверка результатов работы программы, т.е. осуществляется проверка выходных значений функций, которые очень чувствительны, к каким либо возможным изменениям в программном коде;
- создание запускаемой программы на лету в соответствии с ее оригинальным образом — это позволяет избежать возможности выполнения изменений внесенных в программу, так как они не будут учитываться при ее создании.

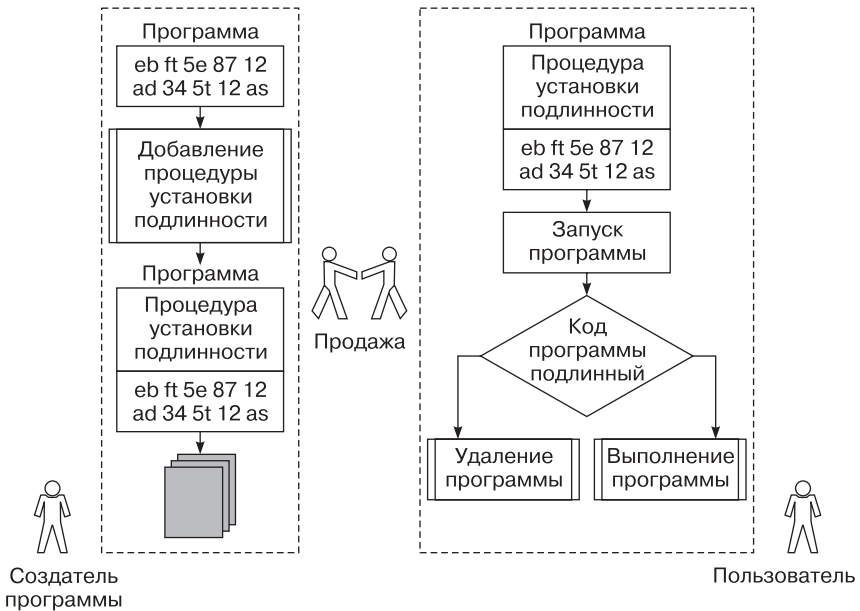


Рис. 4.2. Установка подлинности кода

Шифрование программного кода. Используется для того, чтобы предотвратить вмешательство в программу, а также усложнить процесс понимания взломщиком того, как устроена программа, как она работает, как в ней реализован метод защиты и т.д.

Данный метод защиты предусматривает шифрование кода программы, после чего она в зашифрованном виде поставляется конечным пользователям (иногда эффективно зашифровывать только наиболее важные, критические, участки кода, а не весь код программы). Когда пользователь запускает такую программу, вначале будет запущена процедура расшифровки программы, которой потребуется ключ, с помощью которого и будет расшифрована запускаемая программа (рис. 4.3).

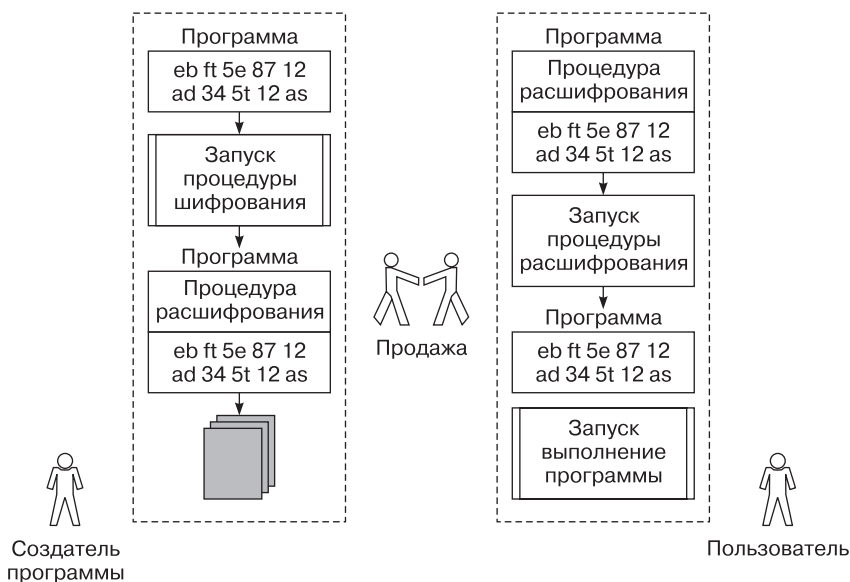


Рис. 4.3. Шифрование программного кода

Способ шифрования программ имеет недостатки, одним из которых является то, что у взломщика есть возможность после приобретения лицензионной копии программы произвести извлечение расшифрованных частей программы в процессе ее работы из памяти. Поэтому сразу после исполнения расшифрованного кода его необходимо выгружать из памяти.

Обфускация программного кода. Обфускация (*obfuscation* — запутывание), это один из методов защиты программного кода, который позволяет усложнить процесс реверсивной инженерии кода защищаемого программного продукта.

Суть процесса обфускации заключается в том, чтобы запутать программный код и устранить большинство логических связей в нем, т.е. трансформировать его так, чтобы он был очень труден для изучения и модификации посторонними лицами (будь то взломщики или программисты которые собираются узнать уникальный алгоритм работы защищаемой программы).

Использование только обфускации для обеспечения полной и эффективной защиты программных продуктов недостаточно, так как она не предоставляет возможности предотвращения нелегального использования программного продукта. Поэтому обфускацию обычно

используют вместе с одним из существующих методов защиты (например, шифрование программного кода), это позволяет значительно повысить уровень защиты программного обеспечения в целом (рис. 4.4).

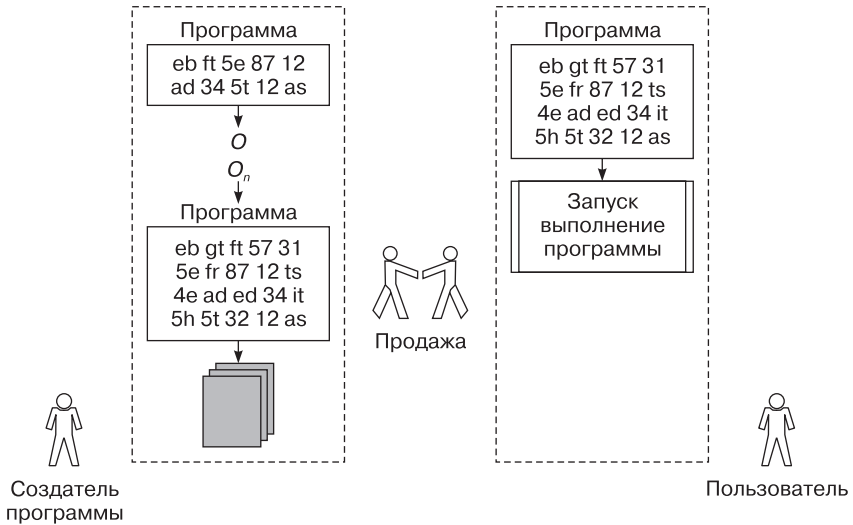


Рис. 4.4. Обфускация программного кода

Так как код, получаемый после осуществления обфускации над одной и той же программой, разный, то процесс обфускации можно использовать для быстрой локализации нарушителей авторских прав (т.е. тех покупателей, которые будут заниматься нелегальным распространением купленных копий программ). Для этого определяют контрольную сумму каждой копии программы, прошедшей обфускацию, и записывают ее вместе с информацией о покупателе в соответствующую базу данных. После этого для определения нарушителя достаточно будет, определив контрольную сумму нелегальной копии программы, сопоставить ее с информацией, хранящейся в базе данных.

Программы-протекторы. Протекторами называются программы, предназначенные для защиты программ от взлома. Данный способ защиты программы стал весьма популярным в последнее время. После того как получен работающий исполняемый файл, этот файл обрабатывается с помощью программы-протектора и создается новый исполняемый файл, в котором реализованы различные средства защиты

(защита программ представляет собой обычную упаковку всего файла, при этом «защищается» сам распаковщик, который в итоге распаковывает файл). Протекторы пишутся профессионалами в области защиты и обеспечивают неплохой уровень противодействия взлому, с которым большинству исследователей программ не справиться.

Недостатком данного метода является то, что протекторы становятся очень популярными и, соответственно, активно изучаются взломщиками, которые пишут программы-антипротекторы (распаковщики защиты). Подобные средства быстро и автоматически снимают защиту.

Использование электронных ключей. На сегодня это наиболее надежный и удобный метод защиты тиражируемого ПО средней и высшей ценовой категории. Он обладает высокой стойкостью к взлому и не ограничивает использование легальной копии программы.

Электронный ключ представляет собой небольшое устройство, которое подсоединяется к одному из портов компьютера (COM, LPT, USB). Принцип его действия состоит в следующем: ключ присоединяется к определенному интерфейсу компьютера, далее защищенная программа через специальный драйвер отправляет ему запрос, который обрабатывается в соответствии с заданным алгоритмом и возвращается обратно. Если ответ ключа правильный, то программа продолжает свою работу. В противном случае она может выполнять любые действия, заданные разработчиками (например, переключаться в демонстрационный режим, блокируя доступ к определенным функциям). Вот некоторые характерные запросы:

- проверка наличия подключения ключа;
- считывание с ключа необходимых программе данных в качестве параметра запуска;
- запрос на расшифрование данных или исполняемого кода, необходимых для работы программы (предварительно разработчик защиты шифрует часть кода программы, и при этом непосредственное выполнение такого зашифрованного кода приводит к ошибке);
- проверка целостности исполняемого кода путем сравнения его текущей контрольной суммы с оригинальной контрольной суммой, считываемой с ключа;
- запрос к встроенным в ключ часам реального времени (при их наличии) и т.д.

Специфика PE-файлов. Типичный исполняемый PE-файл имеет следующую структуру (рис. 4.5).

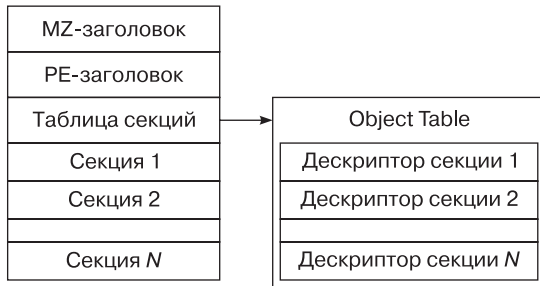


Рис. 4.5. Обобщенная структура PE-файла

MZ-заголовок содержит программу для ОС DOS — эта программа называется *stub* и нужна для совместимости со старыми ОС. Если мы запускаем PE-файл под ОС DOS или OS/2, она выводит на экран консоли текстовую строку, которая информирует пользователя, что данная программа не совместима с данной версией ОС: *This program cannot be run in DOS mode*. После этой DOS-программы идет структура, которая называется `IMAGE_NT_HEADERS` (PE-заголовок). Она описывается следующим образом:

```
typedef struct _IMAGE_NT_HEADERS
{
    DWORD Signature;
    IMAGE_FILE_HEADER FileHeader;
    IMAGE_OPTIONAL_HEADER32 OptionalHeader;
}
```

Первый элемент `IMAGE_NT_HEADERS` — сигнатура PE-файла, которая равна `0x00004550` или в ASCII-символах, «PE00». Кстати, именно из-за этой сигнатуры исполняемый файл Windows и называют PE-файлом.

Далее идет структура, которая называется файловым заголовком и определена как `IMAGE_FILE_HEADER`. Файловый заголовок содержит наиболее общие свойства для данного PE-файла — тип процессора, для которого скомпилирован исполняемый файл; количество заголовков секций; размер опционального заголовка, а также некоторую другую информацию.

После файлового заголовка идет опциональный заголовок — `IMAGE_OPTIONAL_HEADER32`. Он содержит специфические

параметры данного PE-файла — размер исполняемого кода; относительный виртуальный адрес (RVA — Relative Virtual Address) точки входа в программу; базовый адрес, начиная с которого в память будет отображен образ исполняемого файла и многое другое. Также здесь хранится немаловажная информация о границах выравнивания секций в памяти — *SectionAlignment* и границах выравнивания секций в файле на диске — *FileAlignment*. Наличие двух видов выравнивания обусловлено спецификой работы компьютера — чтение информации с жесткого диска производится секторами, а из памяти — страницами. То есть если файл занимает 1 байт, с диска будет все равно прочитано 512 (один сектор). Поэтому размер файла на диске дополняется нулями (выравнивается) до определенного значения (обычно 512). То же самое в памяти, только там выравнивание обычно происходит на одну страницу (1000h). Проще говоря, выравнивание — это округление до определенной константы, нужное для оптимизации работы компьютера.

Прежде чем перейти к дальнейшему описанию, нужно прояснить одну важную вещь — PE-файл состоит из секций. Одна секция может содержать код, данные, таблицу импорта или что-либо еще. Главная сложность заключается в том, что секция на диске не обязательно соответствует секции в памяти. Это происходит по нескольким причинам: отчасти из-за выравнивания, о котором было сказано выше, а еще потому, что существует такая вещь, как неинициализированные данные. К примеру, если в листинге написать: `buffer rb 1000h`, то размер файла на диске не увеличится ни на байт. Другое дело в памяти, здесь он будет больше ровно на 1000h. Информация о секциях хранится в специальной таблице (Object Table). Она идет сразу же после PE-заголовка и состоит из набора заголовков секций. Каждый заголовок описывает одну секцию файла: ее размер, размещение на диске и в памяти, параметры и т.д. Все заголовки секции имеют длину 40 байт и располагаются без выравнивания.

За всеми заголовками в файле следуют тела секций. В Microsoft исторически сложились такие устойчивые названия секций: `.text`, `.bss`, `.data`, `.rdata`, `.edata`, `.idata`, `.rsrc`, `.debug` и некоторые другие. Однако совсем не обязательно, чтобы секции имели такие названия. Например, компиляторы от фирмы Inprise (бывшая Borland) присваивают секциям имена вида `CODE`, `DATA` и т.п. Кроме того, программист может создавать дополнительные секции со своими названиями. Наличие точки в начале секции также необязательно. Секции расположены вплотную друг к другу без промежутков, но это совсем не значит, что в самих сек-

циях не бывает пустых мест. Каждая стандартная секция имеет определенное назначение, например:

- `v.text` расположен код программы;
- `v.bss` размещаются неинициализированные данные;
- `v.data` — инициализированные данные;
- `v.edata` — функции, экспортируемые файлом;
- `v.idata` — таблицы импортируемых функций;
- `v.rsrc` — ресурсы;
- в сегменте `.ldata` — данные только для чтения (строки, константы, информация отладочного каталога).

Но это назначение секций не строгое, например `.text` вполне может быть начинен обычными данными, а также экспортируемыми функциями. Некоторые секции имеют особый формат.

В самом конце PE-файла за секциями вполне могут размещаться дополнительные данные, например какая-нибудь отладочная информация, но это носит необязательный характер.

Техника внедрения кода в PE-файлы. Существуют следующие способы внедрения кода в PE-файлы:

- а) размещение кода поверх оригинальной программы (также называемое затиранием);
- б) размещение кода в свободном месте программы (интеграция);
- в) дописывание кода в начало, середину или конец файла с сохранением оригинального содержимого;
- г) размещение кода вне основного тела файла-носителя (например, в динамической библиотеке).

Поскольку, способ а) приводит к необратимой потере работоспособности исходной программы и реально применяется только в вирусах, здесь он не рассматривается. Все остальные алгоритмы внедрения полностью или частично обратимы.

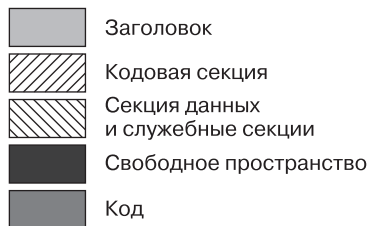


Рис. 4.6. Условные графические обозначения, принятые в дальнейшем изложении

Классификация механизмов внедрения. Механизмы внедрения можно классифицировать по-разному: по месту (начало, конец, середи-

на), «геополитике» (затирание исходных данных, внедрение в свободное пространство, переселение исходных данных на новое место обитания), надежности (предельно корректное, вполне корректное и крайне некорректное внедрение), рентабельности (рентабельное или нерентабельное) и т.д. Следующая классификация основана на характере воздействия на физический и виртуальный образ программы. Все существующие механизмы внедрения можно разделить на четыре категории.

К *категории А* относятся механизмы, не вызывающие изменения адресации ни физического, ни виртуального образов. После внедрения в файл ни его длина, ни количество выделенной при загрузке памяти не изменяется и все базовые структуры остаются на своих прежних адресах. Этому условию удовлетворяют: внедрение в пустое место файла (РЕ-заголовок, хвосты секций, регулярные последовательности), а также внедрение путем сжатия части секции.

К *категории В* относятся механизмы, вызывающие изменения адресации только физического образа. После внедрения в файл его длина увеличивается, однако количество выделенной при загрузке памяти не изменяется и все базовые структуры проецируются по тем же самым адресам, однако их физические смещения изменяются, что требует полной или частичной перестройки структур, привязывающихся к своим физическим адресам, и если хотя бы одна из них останется не скорректированной (или будет скорректирована неправильно), файл-носитель с высокой степенью вероятности откажет в работе. Категории В соответствуют: раздвижка заголовка, сброс части оригинального файла в оверлей и создание своего собственного оверлея.

К *категории С* относятся механизмы, вызывающие изменения адресации как физического, так и виртуального образов. Длина файла и выделяемая при загрузке память увеличиваются. Базовые структуры могут либо оставаться на своих местах (т.е. изменяются лишь смещения, отсчитываемые от конца образа (файла)), либо перемещаться по страничному имиджу произвольным образом, требуя обязательной коррекции. Этой категории соответствует: расширение последней секции файла, создание своей собственной секции и расширение срединных секций.

К «засекреченной» *категории Z* относятся механизмы, вообще не затрагивающие файла-носителя и внедряющиеся в его адресное пространство косвенным путем, например модификацией ключа реестра, ответственного за автоматическую загрузку динамических библиотек. Эту технологию в первую очередь используют сетевые черви и шпионы.

Категория А: внедрение в пустое место файла. Проще всего внедриться в пустое место файла. На сегодня таких мест известно три:

- 1) PE-заголовок;
- 2) хвостовые части секций;
- 3) регулярные последовательности.

Рассмотрим их подробнее.

Внедрение в PE-заголовок

Типичный PE-заголовок вместе с MS-DOS заголовком занимает порядка $\sim 300h$ байт, а минимальная кратность выравнивания секций составляет $200h$ байт. Таким образом, между концом заголовка и началом первой секции практически всегда имеется $\sim 100h$ байт, которые можно использовать для внедрения скрытого сообщения, размещая здесь либо всю внедряемую информацию целиком, либо ее часть, если размер не позволяет большего (рис. 4.7).

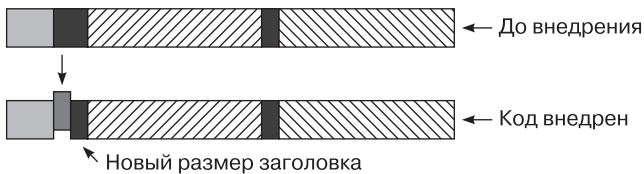


Рис. 4.7. Внедрение информации в свободное пространство хвоста PE-заголовка

Перед внедрением в заголовок необходимо убедиться, что хвостовая часть заголовка действительно свободна. Сканирование таких заголовков обычно выявляет длинную цепочку нулей, расположенных в его хвосте и, очевидно, никак и никем не используемых. Туда-то и можно записать информацию, но только с предосторожностями. Необходимо отсчитать по меньшей мере $10h$ байт от последнего ненулевого символа, оставляя этот участок нетронутым (в конце некоторых структур присутствует до $10h$ нулей, искажение которых ни к чему хорошему не приведет).

Внедрение в PE-заголовок в большинстве случаев можно распознать визуально. Рассмотрим, как выглядит в hex-редакторе типичный исполняемый файл `notepad.exe` (рис. 4.8): как уже было сказано выше, вслед за концом MS-DOS заголовка, обычно содержащим в себе строку `This program cannot be run in DOS mode`, расположена «PE» сигнатура, за которой следует немного мусора, разбавленного нулями и плавно перетекающего в таблицу секций, содержащую легко узнаваемые имена `.text`, `.rsrc` и `.data`.

Внедрение в хвост секции

Операционная система Windows требует, чтобы физические адреса секций были выровнены по меньшей мере на 200h байт, поэтому между секциями практически всегда есть некоторое количество свободного пространства, в которое легко можно внедрить информацию (рис. 4.10).

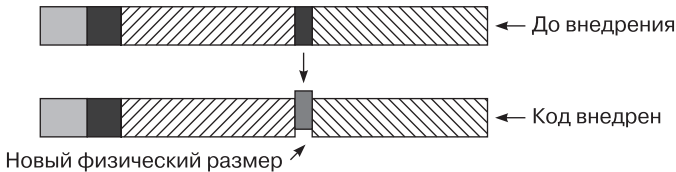


Рис. 4.10. Внедрение кода в хвост секции, оставшийся от выравнивания

Перед внедрением необходимо найти секцию с подходящими атрибутами и достаточным свободным пространством в конце или сосредоточить внедряемый код в нескольких секциях. Для этого необходимо просканировать хвостовую часть секции на предмет наличия непрерывной цепочки нулей — если таковая там действительно присутствует, ее можно безбоязненно использовать для внедрения.

Распознать внедрение этого типа достаточно трудно, особенно если код полностью помещается в первой кодовой секции файла, которой, как правило, является секция .text. Внедрение в секцию данных разоблачает себя наличием характерного кода в ее хвосте.

Внедрение осмысленного машинного кода в хвост секции данных представлено на рис. 4.11.



Внедрение в регулярную последовательность байт

Цепочки нулей необязательно искать в хвостах секций, также как необязательно искать именно нули, для внедрения подходит любая регулярная последовательность (например, цепочка FF FF FF.. или даже FF 00 FF 00...). Если внедряемых цепочек больше одной, код придется разбить по всему телу файла. Соответственно стартовые адреса и длины этих цепочек придется где-то хранить для их последующего восстановления.

Регулярные последовательности чаще всего обнаруживаются в ресурсах, а точнее — в bitmap'ах и иконках. Технически внедрить сюда ничего не стоит, но пользователь тут же заметит искажение иконки, чего допускать ни в коем случае нельзя (даже если это и не главная иконка приложения, так как проводник показывает остальные по нажатию кнопки «сменить значок» в меню свойств ярлыка). Внедрение кода в регулярные цепочки продемонстрировано на рис. 4.12.

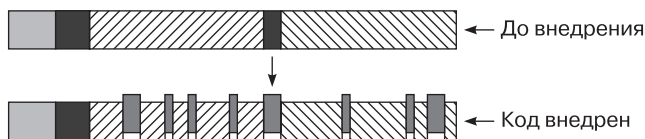


Рис. 4.12. Внедрение кода в регулярные цепочки

Алгоритм внедрения выглядит так: сканируем файл на предмет поиска регулярных последовательностей и отбираем среди них цепочки наибольшей длины, причем сумма их длин должна несколько превышать размеры кода, так как на каждую цепочку в среднем приходится 11 байт служебных данных: четыре байта на стартовую позицию, один байт — на длину, один — на оригинальное содержимое и еще пять байт на машинную команду перехода к другой цепочке.

Разбиваем код на части, добавляя в конец каждой из них команду перехода на начало следующей. Запоминаем начальные адреса, длины и исходное содержимое всех цепочек в импровизированном хранилище, сооруженном либо внутри PE-заголовка, либо внутри одной из цепочек. Если этого не сделать, потом будет невозможно извлечь код из файла. После чего записываем код в выбранные цепочки.

Категория А: внедрение путем сжатия части файла

Внедрение в регулярные последовательности фактически является разновидностью более общей техники внедрения в файл

путем сжатия его части, в данном случае осуществляемое по алгоритму RLE. Если же использовать более совершенные алгоритмы (например, Хаффмена или LZW), то стратегия выбора подходящих частей значительно упрощается. Мы можем сжать кодовую секцию, а на освободившееся место записать наш код. Для компрессии можно использовать функционал, реализованный в самой ОС (аудио, видео-кодеки, экспортеры графических форматов, сетевые функции сжатия и т.д.).

Естественно, упаковка оригинального содержимого секции (или ее части) не обходится без проблем. Во-первых, следует убедиться, что секция вообще поддается сжатию. Во-вторых, предотвратить сжатие ресурсов, таблиц экспорта (импорта) и другой служебной информации, которая может присутствовать в любой подходящей секции файла и кодовой секции в том числе. В-третьих, перестроить таблицу перемещаемых элементов (если, конечно, она вообще есть), исключая из нее элементы, принадлежащие сжимаемой секции и поручая настройку перемещаемых адресов непосредственно самому внедряемому коду (рис. 4.13).

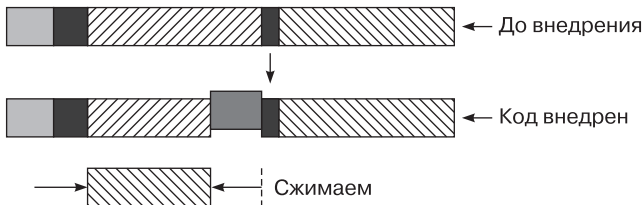


Рис. 4.13. Внедрение кода путем сжатия секции

Распознать факт внедрения в файл путем сжатия части секции трудно, но все-таки возможно. Дизассемблирование сжатой секции обнаруживает некоторое количество бессмысленного мусора,стораживающего опытного исследователя, но зачастую ускользающего от новичка. Также стоит обратить внимание на размеры секций. Если виртуальные размеры большинства секций много больше физических, то файл, по всей видимости, сжат каким-либо упаковщиком.

Категория В: раздвижка заголовка

Когда пространства, имеющегося в PE-заголовке (или какой либо другой части файла), оказывается недостаточно для размещения всего кода целиком, мы можем попробовать растянуть заголовок на величину, выбранную по своему усмотрению. До тех пор пока размер за-

головка (SizeOfHeaders) не превышает физического смещения первой секции, такая операция осуществляется элементарно, но вот дальше начинаются проблемы, для решения которых приходится кардинально перестраивать структуру исполняемого файла. Как минимум необходимо увеличить физические адреса начала всех секций на величину, кратную принятой степени выравнивания, прописанной в поле «Физическое выравнивание секций», и физически переместить хвост файла, записав код на освободившееся место.

Максимальный размер заголовка равен виртуальному адресу первой секции, так как заголовок не может перекрываться с содержимым страничного имиджа. Учитывая, что минимальный виртуальный адрес составляет 1000h, а типичный размер заголовка — 300h, мы получаем в свое распоряжение порядка 3 Кбайт свободного пространства, достаточного для размещения практически любого кода (рис. 4.14).

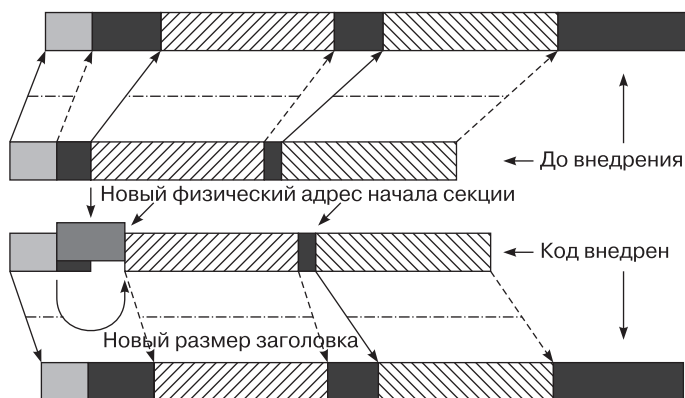


Рис. 4.14. Исполняемый файл и его проекция в память до и после внедрения кода путем раздвижки заголовка

Данный метод внедрения распознается аналогично обычному методу внедрения в PE-заголовок.

Описание программной реализации встраивания ЦВЗ. В программе *Filigrana* использован алгоритм, основанный на встраивании цифрового водяного знака (ЦВЗ) в свободное место исполняемого файла. Подробно данный алгоритм был рассмотрен в предыдущей главе (*Категория А: внедрение в пустое место, в конце секции файла*). В качестве ЦВЗ используется изображение формата BMP.

В общем виде алгоритм встраивания ЦВЗ представлен на рис. 4.15.

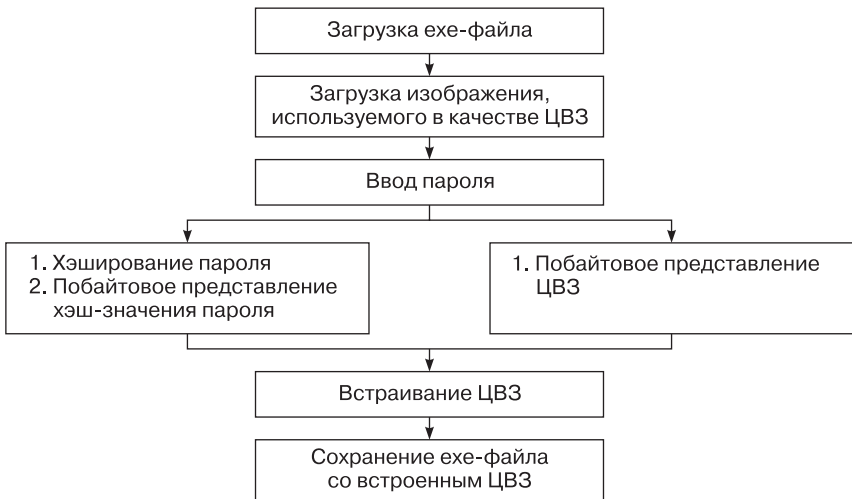


Рис. 4.15. Алгоритм встраивания ЦВЗ

Для демонстрации работы программы встраивания ЦВЗ возьмем два исполняемых файла:

- 1) программу Calc. exe из стандартной поставки Windows, размер программы на диске — 114 Кб (116 376 байт);
- 2) установочный файл SkypeSetup.exe программы Skype, размер файла на диске — 1,88 МВ (1 978 368 байт).

Встроим ЦВЗ наибольшего допустимого размера для каждого из исполняемых файлов.

В нашем случае для первого файла этот размер оказался равным 361 байт, о чем нам и сообщает программа (рис. 4.16).

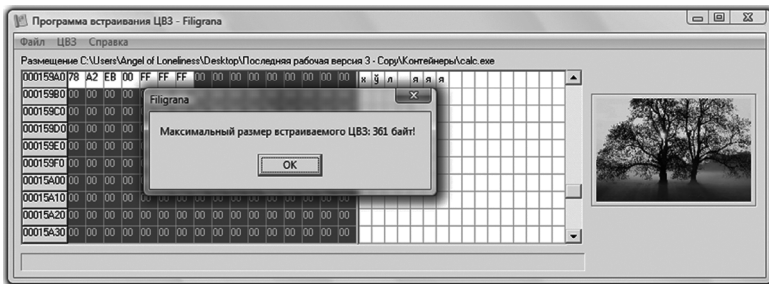


Рис. 4.16. Сообщение о допустимом размере встраиваемого ЦВЗ

Для второго файла возможный размер ЦВЗ для встраивания увеличился до 13 668 байт (рис. 4.17).

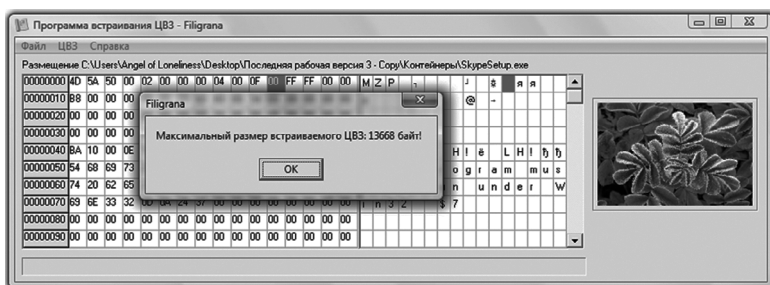


Рис. 4.17. Сообщение о допустимом размере встраиваемого ЦВЗ

При этом в процентном соотношении (размер встраиваемого ЦВЗ к размеру исполняемого файла) первый файл выигрывает у второго $\sim 0,3\%$ против $\sim 0,15\%$. Но, как видно из выходных данных, наиболее эффективно использование программы для встраивания в файлы большего размера, что логично с точки зрения структуры исполняемых файлов, да и выявить факт встраивания в exe-файл большего размера будет сложнее из-за размеров дизассемблированного кода, при этом рекомендуется использовать ЦВЗ небольшого размера относительно размеров контейнера.

После встраивания размеры исполняемых файлов не изменились (рис. 4.18).

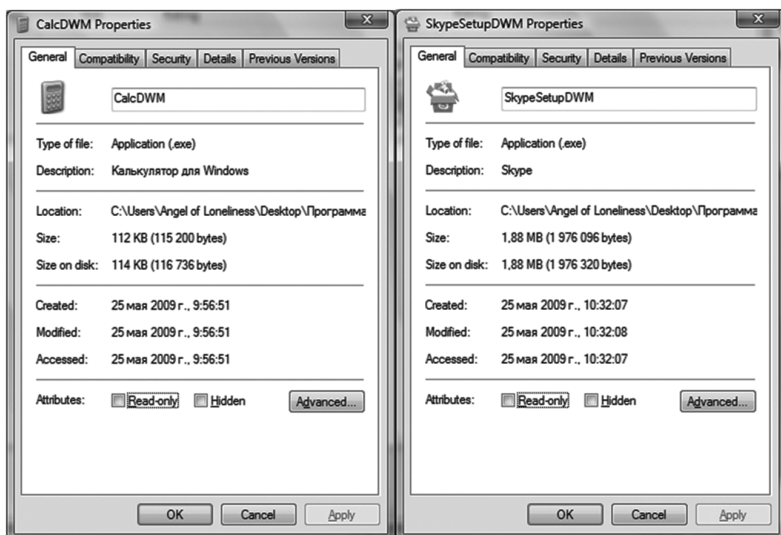
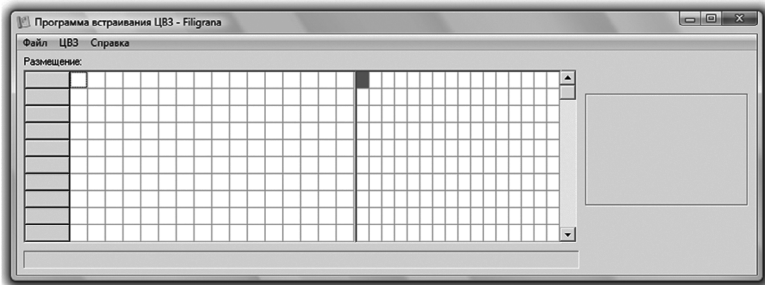


Рис. 4.18. Размеры exe-файлов, содержащих ЦВЗ

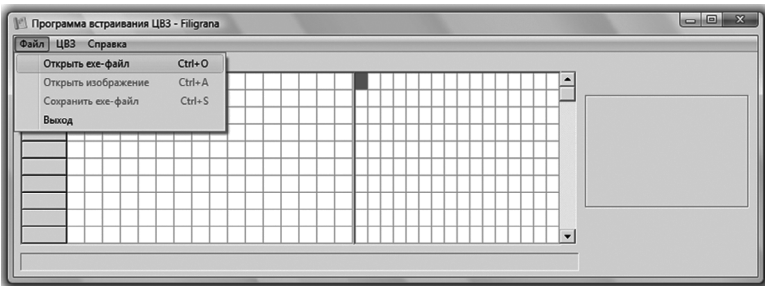
Задание

Встраивание цифрового водяного знака

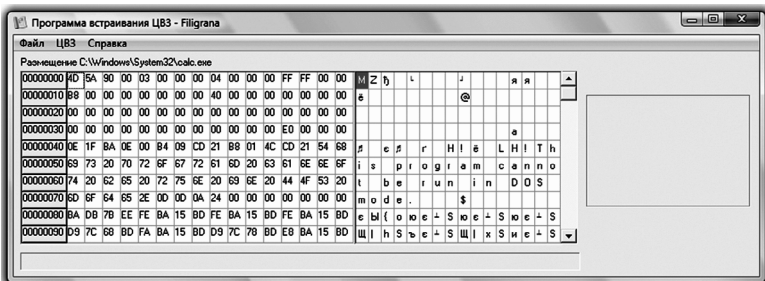
1. Запустите программу *Filigrana*, предварительно установленную на ваш компьютер.



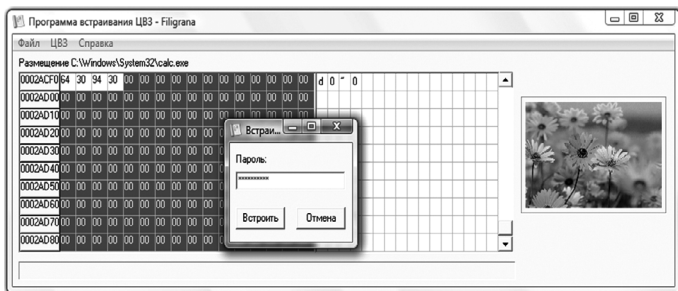
2. Выберите пункт «Открыть ехе-файл» из меню «Файл» появившегося окна или нажмите комбинацию клавиш «Ctrl + O».



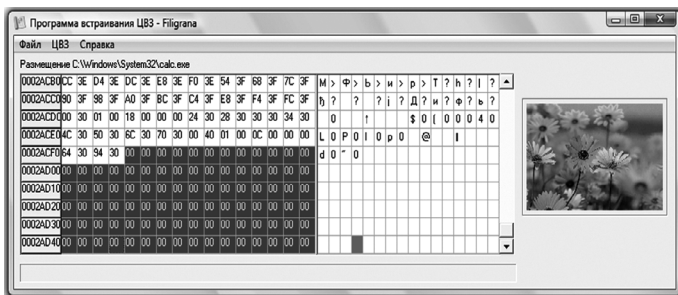
3. В появившемся диалоговом окне выберите необходимый ехе-файл, два раза кликнув на нем левой кнопкой мыши или с помощью кнопки «Открыть». В окне главной формы отобразятся данные о выбранном файле в шестнадцатеричном и ASCII формате.



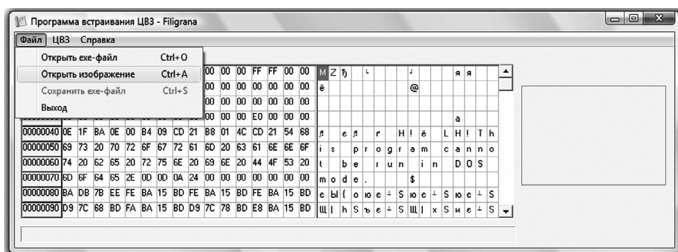
4. Выберите пункт «Открыть изображение» из меню «Файл» или нажмите комбинацию клавиш «Ctrl + A».



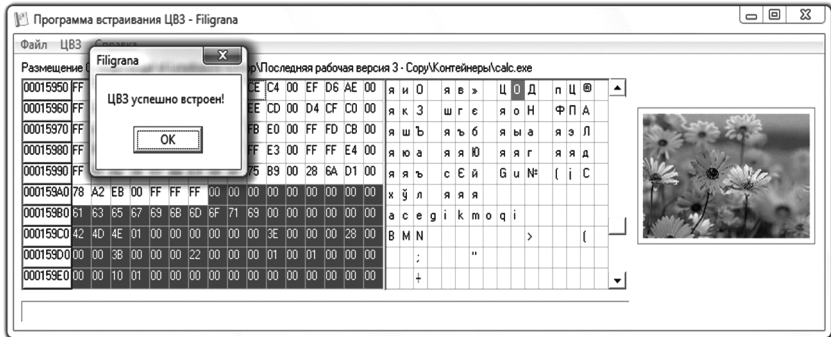
5. В появившемся диалоговом окне выберите изображение в формате .bmp, которое вы хотите использовать в качестве ЦВЗ, два раза кликнув на нем левой кнопкой мыши или с помощью кнопки «Открыть». В окне preview главной формы отобразится выбранное изображение, в hex-окне курсор автоматически переместится на строку, с которой начнется последующее встраивание цифрового водяного знака.



6. Чтобы встроить выбранное изображение, введите пароль (используйте сочетание букв, цифр и символов) в окне «Встраивание ЦВЗ», вызываемого из пункта «Встроить изображение» меню «ЦВЗ» главной формы или нажатием комбинации клавиш «Alt + I».



7. После ввода пароля нажмите кнопку «Встроить». В случае если выбранное изображение удовлетворяет условиям встраивания (размер ЦВЗ не превышает размер контейнера и ехе-файл пригоден для встраивания), оно будет встроено, после чего появится соответствующее сообщение, а в hex-окне можно будет просмотреть результаты встраивания.

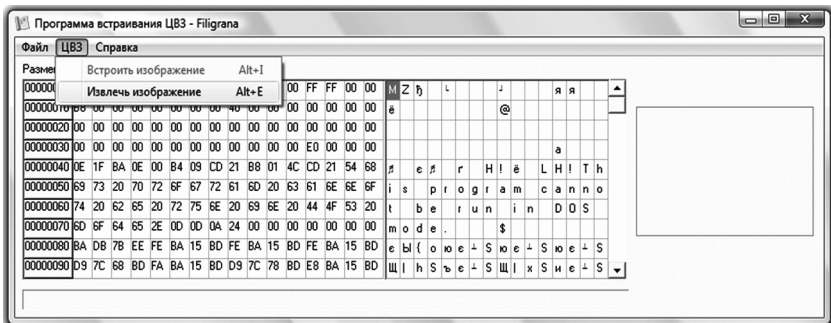


8. Для сохранения исполняемого файла со встроенным ЦВЗ выберите пункт «Сохранить ехе-файл» из меню «Файл» или нажмите комбинацию «Ctrl + S», введите имя сохраняемого файла и нажмите кнопку «Сохранить».

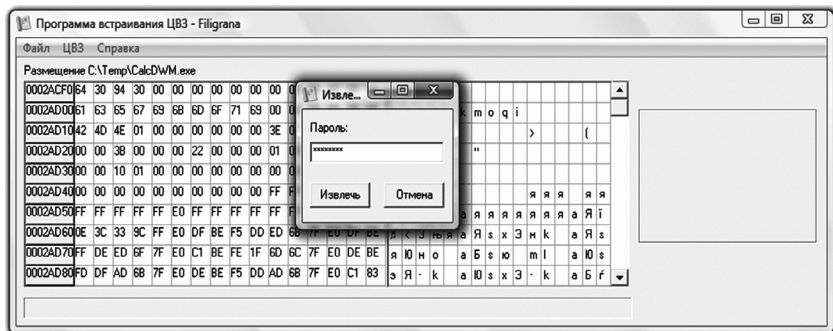
Внимание! Для успешного извлечения ЦВЗ необходимо запомнить свой пароль или сохранить его в месте, недоступном для других пользователей. Никому не показывайте свой пароль!

Извлечение цифрового водяного знака

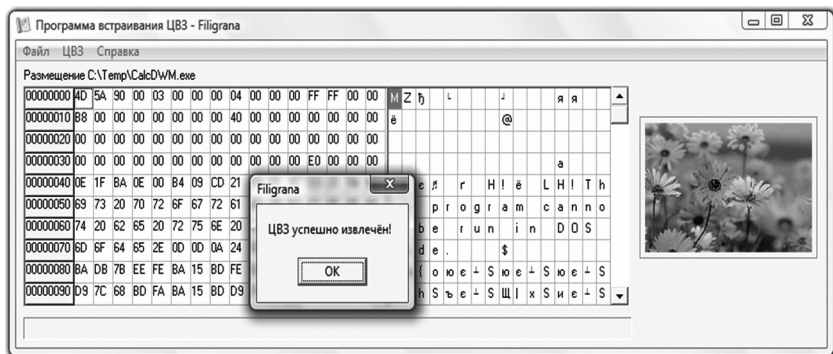
1. Откройте ехе-файл, содержащий ЦВЗ. Выберите пункт «Извлечь изображение» меню «ЦВЗ» главной формы или нажмите комбинацию клавиш «Alt + E».



2. Для извлечения изображения в окне «Извлечение ЦВЗ» введите пароль, который использовался при встраивании цифрового водяного знака, и нажмите кнопку «Извлечь».



3. Если Вы ввели верный пароль, появится соответствующее сообщение, а в окне preview главной формы отобразится изображение, встроенное ранее.



4. Сохраните в отчете экранные формы, демонстрирующие процесс встраивания ЦВЗ в различные исполняемые файлы. Сделайте выводы об эффективности изученного метода защиты.

5. Включите в отчет о лабораторной работе ответы на контрольные вопросы, выбранные в соответствии с номером варианта из табл. 4.1.

Таблица 4.1

Номер варианта	Контрольные вопросы
1, 5, 7, 26	Перечислите способы защиты программных продуктов. Укажите их достоинства и недостатки
2, 4, 6	Какие методы включает юридическая защита программных продуктов? Охарактеризуйте основные из них
11, 13	Перечислите основные международные и отечественные источники защиты прав авторов программ
12, 14, 16	В чем заключается процедура лицензирования программ? Какими нормативными документами регулируется процесс лицензирования программ?
3, 9, 18, 29	Перечислите технические методы защиты программных продуктов. Кратко охарактеризуйте каждый из них
20, 22, 24	Какие методы стеганографии могут использоваться для защиты программных продуктов? Сравните методы стеганографической защиты и технической защиты ПО
10, 17, 19	Какие особенности структуры РЕ-файлов дают возможность эффективного внедрения цифровых водяных знаков?
21, 23, 25	Опишите суть метода внедрения кода в РЕ-файлы за счет размещения кода в свободном месте программы (интеграция)
8, 28, 27	Какой из методов внедрения кода в РЕ-файлы используется в программе <i>Filigrana</i> ? Опишите суть этого метода, его достоинства и недостатки
12, 15, 30	Какие способы обнаружения, извлечения и модификации ЦВЗ вы можете предложить для изученного метода защиты ПО?